SOFTWARE ENGINEERING

Lab 7: Fuzzing for Input Validation Bugs

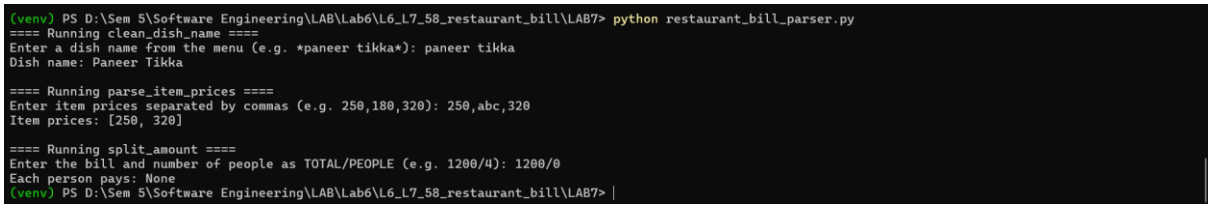
Name: Sakshi Srinivas Ghodke

SRN: PES2UG24CS914

Date: 09/11/2026

Repo Link:
https://github.com/annabeth12124stack/L6_L7_58_restaurant_bill/tree/main/LAB7

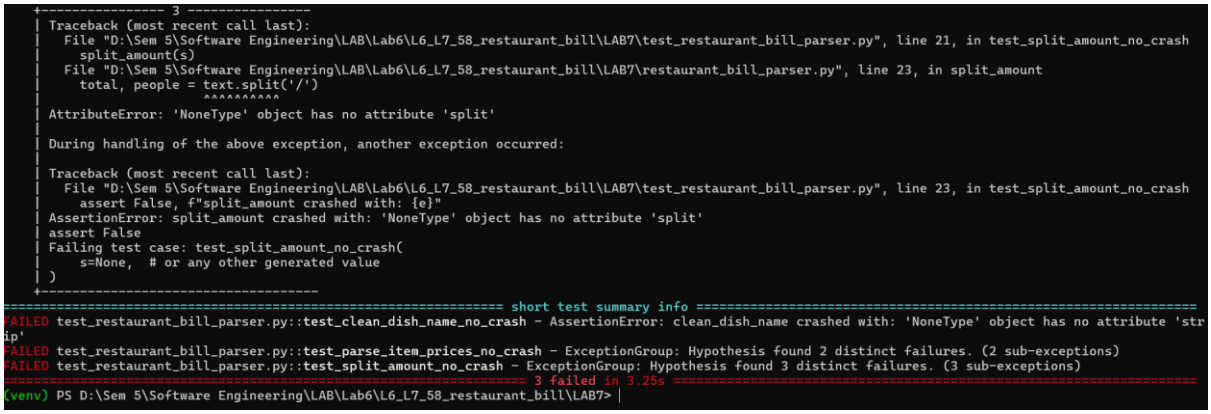
Screenshots:



Running the buggy restaurant_bill_parser.py: entering 250,abc,320 crashes parse_item_prices with a ValueError



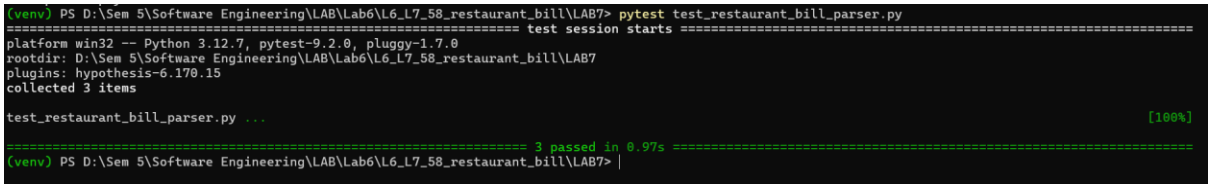
Running pytest on the buggy module: all three Hypothesis tests fail (FFF). Hypothesis finds that clean_dish_name(None) raises an AttributeError



Summary of the failing run: 3 failed, with Hypothesis reporting 1 failure for clean_dish_name, 2 for parse_item_prices and 3 for split_amount.



Running the fixed restaurant_bill_parser.py: the same inputs no longer crash. It returns [250, 320] for the prices and None for 1200/0.



Running pytest on the fixed module: all 3 tests pass (3 passed).

Reflection Questions:

1. What is fuzz testing, and how did Hypothesis find bugs that a normal unit test might miss?
Fuzz testing feeds a program random or unexpected inputs to see whether it crashes or misbehaves. In this lab I only wrote one test per function, which said “this function should never raise an exception”, and Hypothesis generated the inputs itself, including empty strings, None and a lone /. I would not have thought to write those unit tests by hand, because the code assumed every input was well formed. Hypothesis also reports the smallest failing input, which made each bug easy to spot.

2. What bugs did Hypothesis find, and how did you fix them?
It found crashes in all three functions. clean_dish_name(None) raised an AttributeError because None has no .strip(). parse_item_prices crashed on '' with a ValueError (from int('')) and on None with an AttributeError. split_amount crashed on '', '/' and None, either because the string couldn't be unpacked into two values or because int('') failed. Hypothesis never generated 1/0 in my run, but reading the code showed that it would raise ZeroDivisionError, so I handled it too. I fixed all of these with two techniques: an isinstance(..., str) check at the start of each function, and a try/except ValueError around int() instead of a bare except. Invalid input now gives a safe result ("", [] or None) instead of an exception.

3. What are the limitations of this kind of test, and what would you add?
My tests only check that the functions don't crash, so they can't tell whether the answer is correct. A function that returned None for every input would pass. My parse_item_prices also silently skips entries like abc, which could hide a mistake on a real bill, and returning an empty list for the whole input might be safer. To strengthen this, I would add properties that check the output, for example that the result is always a list of integers, or that split_amount("1200/4") equals 300. I would also keep a few normal unit tests for known inputs, since fuzzing complements them but doesn't replace them.